Telkom 2. ZH: Socket

Magyarázó + offline gyakorló csomag | Python, TCP, UDP, struct, select, proxy

Röviden: a ZH egy kis számú, sablonos feladat. Nem kitalálni kell, hanem öt mintát gépiesen, hibamentesen megírni. Ez a csomag mind az öt mintát külön futtatható példán mutatja meg, utána te írod meg újra őket üres vázból, a gép pedig ellenőrzi.

Mi van a csomagban

Mappa	Tartalom	Mire jó
`magyarazat/`	ez a PDF	megérteni a lényeget, utánanézni, ha elakadsz
`peldak/`	`p1` ... `p5` (futtatható, kommentelt)	megnézed, lefuttatod, megérted
`feladatok/`	`f1` ... `f5` (üres vázak + önellenőrző teszt)	te írod meg, a teszt azonnal szól, ha jó
`feladatok/megoldasok/`	kész megoldások	csak akkor nézd, ha tényleg elakadtál

Javasolt menet (pomodoro blokkok, kb. 25 perc + 5 perc szünet)

Blokk	Mit csinálsz	Fájl
1.	PDF 1-3. fejezet: alapok, struct, recv_exact	futtasd: `p1`
2.	PDF 4. fejezet, majd írd meg magadtól az 1. feladatot	`f1`
3.	PDF 5. fejezet (select), majd a select-es szerver	`p3`, `f2`
4.	PDF 6/a és 6/b: UDP szerver, ellenőrző kliens	`p2`, `p4`, `f3`, `f4`
5.	PDF 6/c: proxy	`p5`, `f5`
6.	Saját segédlet összeállítása a saját megoldásaidból (9. fejezet), hibaüzenet-kereső átfutása	-

TIPP Ha csak 1 órád van: 1-3. fejezet, `p1` és `p3`, majd `f1` és `f2`. A 3. feladat ezekre épül (a proxy egy select-es szerver + egy UDP kliens kombinációja), ezért ez az alap.

FONTOS A ZH nyílt könyves, de az AI használata tilos. Az, hogy AI-val készült segédletet be szabad-e vinni, **nincs benne a feltöltött anyagban**: kérdezd meg a gyakvezetőt. A biztos út, hogy a segédletedet a saját, kézzel megírt megoldásaidból állítod össze (lásd 9. fejezet). Ez tanulásnak is jobb. Ezt a csomagot tanuláshoz használd.

0. A ZH egy oldalon

Forrás: a Moodle-ről mentett Minta ZH (Socket). A ZH-n a konkrét feladatok ennek változatai lehetnek, ezért ne a számokat és parancsneveket magold, hanem a mintákat.

Feladat	Mit kell írni	Formátumok	Példa	Nehézség
1. TCP kliens	kliens; a szerver adott. a) input, b) txt, c) json, d) UDP	`20s i` küld, `Xs` kap	`p1`	könnyű
2. select szerver	TCP szerver több klienssel; IN / INCR / DECR	`4s i` kap, bytesztringet küld	`p3`	**a legnehezebb alap**
3/a UDP szerver	PUSH / PLUS / MINUS	`5s i` kap, `i` küld	`p2`	könnyű-közepes
3/b ellenőrző kliens	TCP és UDP értéket összehasonlít	mindkettő	`p4`	könnyű

Feladat	Mit kell írni	Formátumok	Példa	Nehézség
3/c proxy	TCP szerver, ami UDP szerverhez továbbít	`4s i` → `5s i` → `i`	`p5`	közepes-nehéz

Mi nincs az anyagban (kérdezz rá)

- a pontozás és az időkeret (nincs a Moodle-oldalon),
- a ZH konkrét paraméterei (parancsnevek, formátumok, portok változhatnak),
- hogyan éred el a ZH-s tesztkörnyezetet (melyik porton, melyik címen hívja a szerveredet).

A félév áttekintésed szerint a gyakorlati ZH-knál mindegyikből legalább 50% kell, a célszint 3-4-es (Félév áttekintés, TKH szakasz).

A ZH első 3 perce: ezt írd ki magadnak minden feladatnál

- **Ki a szerver, ki a kliens?** Melyiket kell megírni, melyik adott?
- **TCP vagy UDP?** (ettől függ: connect/accept vagy sendto/recvfrom)
- **Mit küld a kliens?** Struct formátum, és hány bájt (`Struct(...).size`).
- **Mit kap vissza?** Bytesztring (`b"24"`) vagy struct (melyik formátum)?
- **Kell-e select?** (több kliens egyszerre)

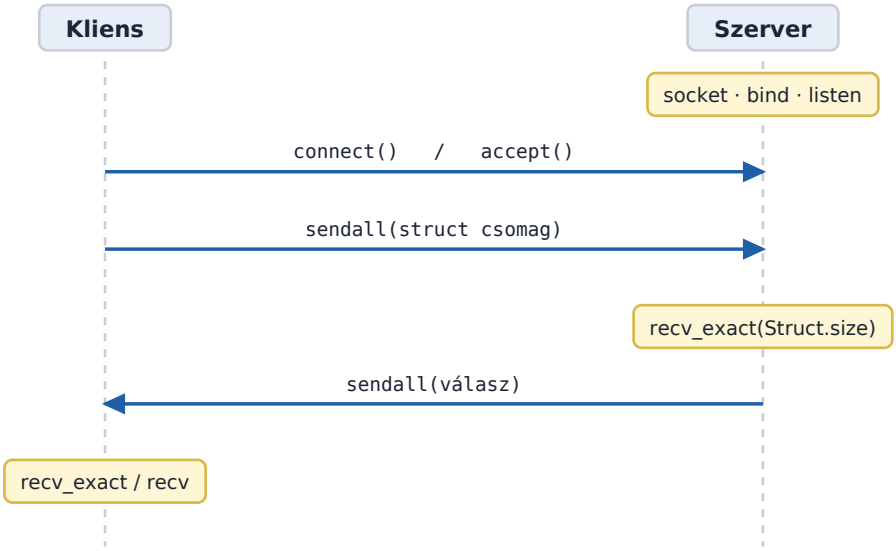
Először mindig a legkisebb működő változatot (egy kliens, egy parancs) írd meg, és futtasd. Utána jöhet a többi.

1. Alapok: mit csinál egy socket?

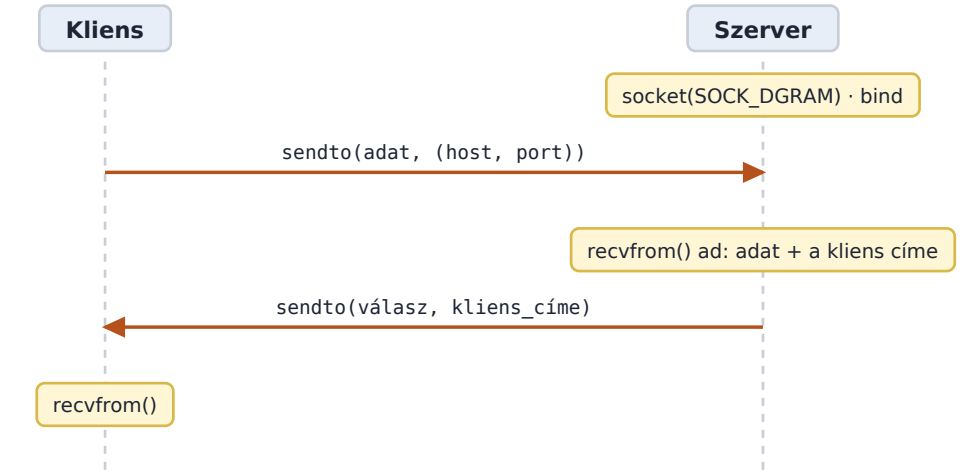
- A **socket** a programod „csatlakozója” a hálózathoz. A címe két dolog: **(host, port)**.
- **host**: melyik gép. Gyakorláskor `"localhost"` (a saját géped).
- **port**: egy szám (pl. 10000). Egy gépen belül ez dönti el, melyik program kapja az adatot.
- Két fajta socket van, ezt a ZH mindkettővel használja: **TCP** és **UDP**.

	TCP (`SOCK_STREAM`)	**UDP (`SOCK_DGRAM`)**
kapcsolat	van: connect / accept	nincs
adat	**bájtfolyam**, nincs üzenethatár	különálló **datagramok**: egy recvfrom = egy teljes üzenet
megbízhatóság	sorrend és teljesség garantált	elveszhet, felcserélődhet (localhoston ritka)
szerver hívásai	`bind, listen, accept, recv, sendall`	`bind, recvfrom, sendto`
kliens hívásai	`create_connection, sendall, recv`	`sendto, recvfrom`

TCP: így zajlik egy kérés-válasz



UDP: nincs kapcsolat, a cím minden csomaggal együtt jön



Legkisebb példa (órai demó: `blocking_client.py`):

```
import socket
with socket.create_connection(("localhost", 10000)) as sock:   # connect
    sock.sendall(b"Hello server")                              # küldés: BÁJTOK, nem str
    print(sock.recv(1024))                                     # a válasz bájtokban jön
```

> **TIPP** A **sendall** és **recv** mindig bájtokat (`bytes`) kezel. Szöveget a `.encode()` alakít bájtokká, a `.decode()` vissza.

2. struct: bájtok összeállítása és kibontása

A hálózaton bájtok mennek, nem számok vagy szövegek. A `struct` modul adott formátum szerint bájtokká csomagol (**pack**), majd visszabontja őket (**unpack**). A kliens és a szerver ugyanazt a formátumot használja, így értik egymást.

Jel	Jelentése	Mérete
`s`	bájtsor fix hosszal: `20s` = pontosan 20 bájt	a szám szerint
`i`	egész szám (int)	4 bájt
`c`	egyetlen bájt	1 bájt

```
import struct
REQ = struct.Struct("20s i")           # 20 bájt szöveg + 4 bájt szám

csomag = REQ.pack(b"almafa", 4)        # a szöveg bájt kell: "almafa".encode() vagy b"almafa"
print(len(csomag), csomag[:10])        # 24  b'almafa\x00\x00\x00\x00'  <- nullákkal kipótolta
szoveg, n = REQ.unpack(csomag)         # unpack mindig TUPLE-t ad vissza
print(szoveg, n)                       # b'almafa\x00\x00 ...' (összesen 20 bájt)  4
print(szoveg.rstrip(b"\x00").decode()) # almafa   <- nullák levágása + vissza szöveggé
```

A négy dolog, amit minden alkalommal meg kell csinálnod

- **Küldés előtt**: szöveg → bájt: `szoveg.encode()`.
- **Fogadás után**: `s` mező → `.rstrip(b"\x00").decode()` (a kipótolt nullák miatt).
- **Az unpack tuple-t ad**: `(x,) = Struct('i').unpack(adat)` vagy `...unpack(adat)[0]`.
- **Dinamikus formátum**: ha a hossz változik, `f"{n}s"` (pl. `"4s"`, `"3s"`).

CSAPDA A méret nem az, aminek gondolod. `struct.Struct("5s i").size` = **12**, nem 9: az 5 bájt szöveg után 3 kitöltő bájt kerül, hogy az `i` 4-gyel osztható helyen kezdődjön. Ezért **soha ne írd be kézzel a számot**, mindig `Struct.size`. Ráadásul ha elé írsz egy `!`, `<` vagy `>` jelet, a méret megváltozik (`"!5s i"` = 9 bájt), és a szerver nem értené. A ZH-n pontosan azt a formátumot írd, amit a feladat ad.

Formátum	Méret	Miért
`"20s i"`	24	20 + 4 (a 20 osztható 4-gyel, nincs kitöltés)
`"4s i"`	8	4 + 4
`"5s i"`	12	5 + **3 kitöltés** + 4
`"10s i"`	16	10 + **2 kitöltés** + 4 (órai `tcp_proxy.py`)
`"i i c"`	9	4 + 4 + 1 (órai `select_server.py`)

3. recv_exact: miért nem elég a recv?

A TCP **bájtfolyam**: nincs benne üzenethatár. Ha a kliens két darab 8 bájtos csomagot küld, a szerver `recv(8)` hívása kaphat 8 bájtot, de 5-öt vagy 16-ot is.

```
Kliens küld:                [ 8 bájt ][ 8 bájt ]          (két sendall)
A szerver recv(8)-a kaphat: [ 8 bájt ]   vagy   [ 5 bájt ]   vagy   [ 16 bájt ]
```

Ezt az órai demók is kipróbálják (`select_client.py`: `two_messages` és `split_message`). A megoldás: addig olvasunk, amíg a **pontos** méret meg nem jön:

```
def recv_exact(sock, size):
    data = b""
    while len(data) < size:
        chunk = sock.recv(size - len(data))
        if not chunk:          # üres = a másik oldal bontott
            return b""
        data += chunk
    return data
```

Mit olvasol	Mit használj
fix méretű struct (pl. 8 vagy 12 bájt)	`recv_exact(sock, CMD.size)`
rövid sima bytesztring válasz (pl. `b"24"`)	`sock.recv(1024)` elég, mert kérdés-válasz sorban megy
UDP-n bármi	`recvfrom(1024)`: egy hívás = egy teljes datagram, nem kell recv_exact

TIPP A `recv_exact`-et írd be a segédletedbe készen. Ez az a 8 sor, amit a ZH-n a legjobb nem újra kitalálni.

4. feladat: TCP kliens (`p1_tcp_kliens.py`)

Adott: a szerver. **Neked**: a kliens. A kliens egy (szöveg, szám) párt küld `20s i` formátumban, a szerver pontosan *szám* bájtos választ ad (formátuma: `Xs`).

Folyamat, 5 lépés

```
def tcp_kliens(text, n):
    with socket.create_connection(("localhost", PORT)) as sock:   # 1) kapcsolódás
        sock.sendall(REQ.pack(text.encode(), n))                  # 2) csomagolás + küldés
        valasz = recv_exact(sock, n)                              # 3) pontosan n bájt jön vissza
    (szoveg,) = struct.unpack(f"{n}s", valasz)                    # 4) kicsomagolás: "4s", "3s"...
    return szoveg.rstrip(b"\x00").decode()                       # 5) nullák le, vissza szöveggé
```

CSAPDA A feladat szövege szerint a szerver „levág a szöveg elejéről X karaktert”, de a példák (`almafa,4 -> amla`, `kortefa,3 -> rok`) azt mutatják: az **első X** karaktert tartja meg, és megfordítja. A szerver adott, ezt neked nem kell megírni, csak azt tudd, hogy a válasz hossza pontosan X.

A négy változat: csak a bemenet forrása különbözik

Változat	Bemenet	Mire figyelj
a)	`input()` + `random.randint(1, len(text))`	a feladat nem ad tartományt; a `len(text)` a biztonságos
b)	`input.txt`: soronként `szó szám`	`.read().splitlines()`, `randint(1, 3)` (**mindkét vége benne van**), `sorok[i-1].split()`, a szám `int(...)`!
c)	`input.json`: `{"1": ["almafa", 4], ...}`	a kulcs **szöveg**: `str(random.randint(1, 3))`, `json.load(f)`
d)	UDP kliens	`SOCK_DGRAM`, `sendto` / `recvfrom(1024)`, nincs connect, nincs recv_exact; `settimeout(2)`

Tipikus hibák

- elfelejtett `.encode()` (a `s` mezőnek bájt kell),
- a válaszban ott maradnak a `\x00` bájtok (nincs `rstrip`),
- a fájlból olvasott szám szöveg marad (nincs `int()`),
- a fájl útvonala rossz: a program **abból a mappából** keresi, ahonnan indítod (a példában `os.path.dirname(__file__)`),
- UDP-nél `recv_exact`-et használsz: UDP-n nincs ilyen, `recvfrom` kell.

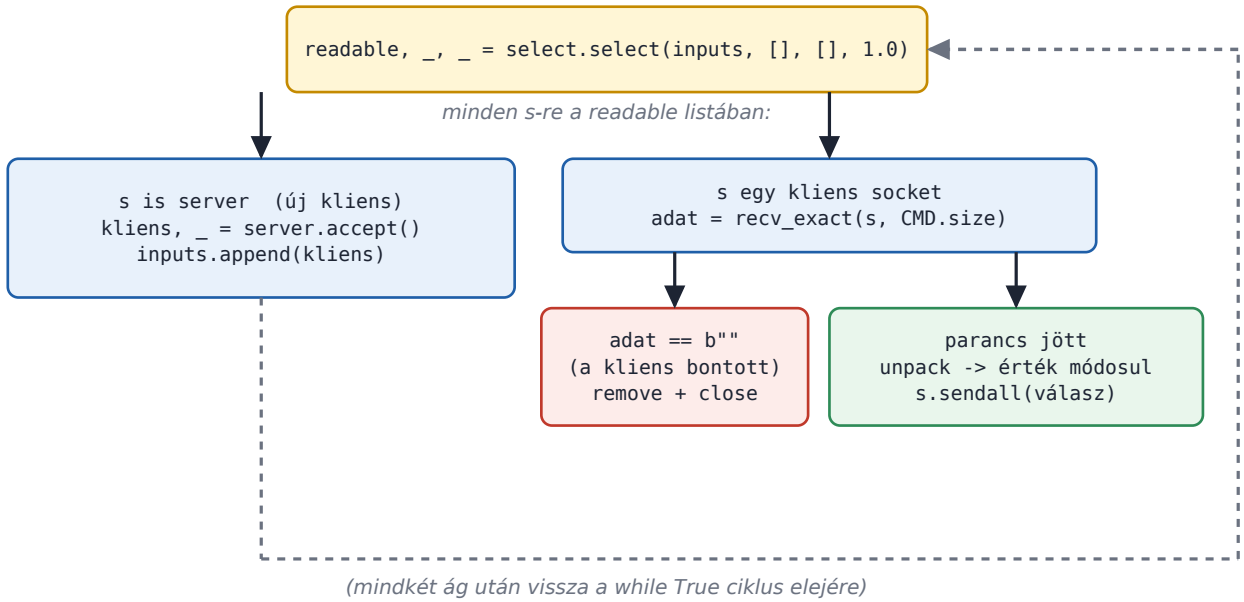
5. feladat: TCP szerver select-tel (`p3_select_tcp_szerver.py`)

Feladat: a szerver egy egész számot tárol (kezdetben 0). Parancsok `4s i` formátumban: `IN x` (beállít), `INCR x` (növel), `DECR x` (csökkent). Minden parancs után az új értéket **sima bytesztringként** küldi vissza. Példa: `IN 20` → `b"20"`, `INCR 4` → `b"24"`, `DECR 5` → `b"19"`.

Miért kell a select?

Az órai blokkoló szerver (`blocking_server.py`) egyszerre egy klienst szolgál ki: accept → recv → válasz → következő. Ha három kliens egyszerre jön, a többi sorban áll. A **select** azt mondja meg: „melyik socketen van most teendő?” Így egy szál egyszerre sok klienst kezel.

A ciklus (ez a lényeg)




```
ertek = 0                                       # KÖZÖS állapot, a ciklus ELŐTT
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("localhost", PORT)); server.listen(5)
    inputs = [server]                           # a figyelt socketek listája
    while True:
        readable, _, _ = select.select(inputs, [], [], 1.0)
        for s in readable:
            if s is server:                     # 1) új kliens
                kliens, cim = server.accept()
                inputs.append(kliens)
                continue
            try:                                # 2) meglévő kliens
                data = recv_exact(s, CMD.size)
            except ConnectionResetError:        # Windowson előfordulhat bontáskor
                data = b""
            if not data:                        # üres = a kliens bontott
                inputs.remove(s); s.close()
                continue
            nev, x = CMD.unpack(data)
            nev = nev.rstrip(b"\x00").decode()  # b"IN\x00\x00" -> "IN"
            if nev == "IN":     ertek = x
            elif nev == "INCR": ertek += x
            elif nev == "DECR": ertek -= x
            s.sendall(str(ertek).encode())      # bytesztring: b"24"
```

Soronként, mit miért

- `inputs = [server]`: amit a select figyel. A figyelő socket mindig benne van, az új kliensek bekerülnek.
- `select(inputs, [], [], 1.0)`: a *readable* azokat adja vissza, amelyeken van olvasnivaló. Az 1.0 timeout miatt a ciklus nem ragad be örökre (és Windowson a Ctrl+C is így tud működni, lásd 8. fejezet).
- `s is server`: a figyelő socketen „olvasnivaló” = új kapcsolat érkezett, ezért `accept`.
- **üres adat (`b""`) = a kliens bontott**: ki kell venni a listából és be kell zárni.
- az érték a cikluson **kívül** él, így minden kliens ugyanazt látja (a P3 példa két klienssel bizonyítja).

Tipikus hibák

- `inputs.remove(s)` elfelejtése: a bontott socket örökre „olvasható” marad (üres adat), és a szerver végtelen ciklusban pörög (az órai `select_server.py` [K5] kísérlete pont ezt mutatja).
- az `ertek = 0` a ciklusban van: minden üzenetnél nullázódik.
- a parancsnévnek nincs `rstrip`: a `"IN\x00\x00"` nem egyenlő `"IN"`-nel.
- `sendall(str(ertek))`: str nem megy, `.encode()` kell.
- a figyelő socketen `recv`-et hívsz `accept` helyett.

6. feladat: UDP szerver, ellenőrző kliens, proxy

6/a UDP szerver (`p2_udp_szerver.py`)

Ugyanaz az ötlet, mint a select-es szervernél (egy tárolt szám), de más parancsnevekkel (`PUSH`, `PLUS`, `MINUS`), `5s i` kéréssel, és a válasz itt **struct** (`i`), nem bytesztring.

```
CMD = struct.Struct("5s i")        # 12 bájt! (5 + 3 kitöltés + 4)
REPLY = struct.Struct("i")
ertek = 0                                          # a ciklus előtt
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sock.bind(("localhost", PORT))                 # nincs listen, nincs accept
    while True:
        data, kliens = sock.recvfrom(1024)         # a kliens címe is jön
        nev, x = CMD.unpack(data)
        nev = nev.rstrip(b"\x00").decode()
        if nev == "PUSH":    ertek = x
        elif nev == "PLUS":  ertek += x
        elif nev == "MINUS": ertek -= x
        sock.sendto(REPLY.pack(ertek), kliens)     # struct válasz, a kliens címére
```

CSAPDA UDP-nél a válasz a **feladó címére** megy (`kliens`), nem egy korábban megnyitott kapcsolatra. A választ `REPLY.pack(ertek)`-vel kell csomagolni, nem `str(ertek).encode()`-dal.

6/b Ellenőrző kliens (`p4_ellenorzo_kliens.py`)

Az UDP szerver néha hibázik: minden parancsnál a kapott számhoz hozzáad -1, 0 vagy 1 értéket. Az ellenőrző kliens mindkét szervertől lekérdezi az értéket, és összehasonlítja őket.

- „**0-val való növelés**” = lekérdezés: `INCR 0` a TCP szervernek, `PLUS 0` az UDP szervernek. Nem változtat az értéken (az UDP szerver hibája miatt mégis elcsúszhat).
- ha `tcp_ertek != udp_ertek`, az UDP szerver hibázott.

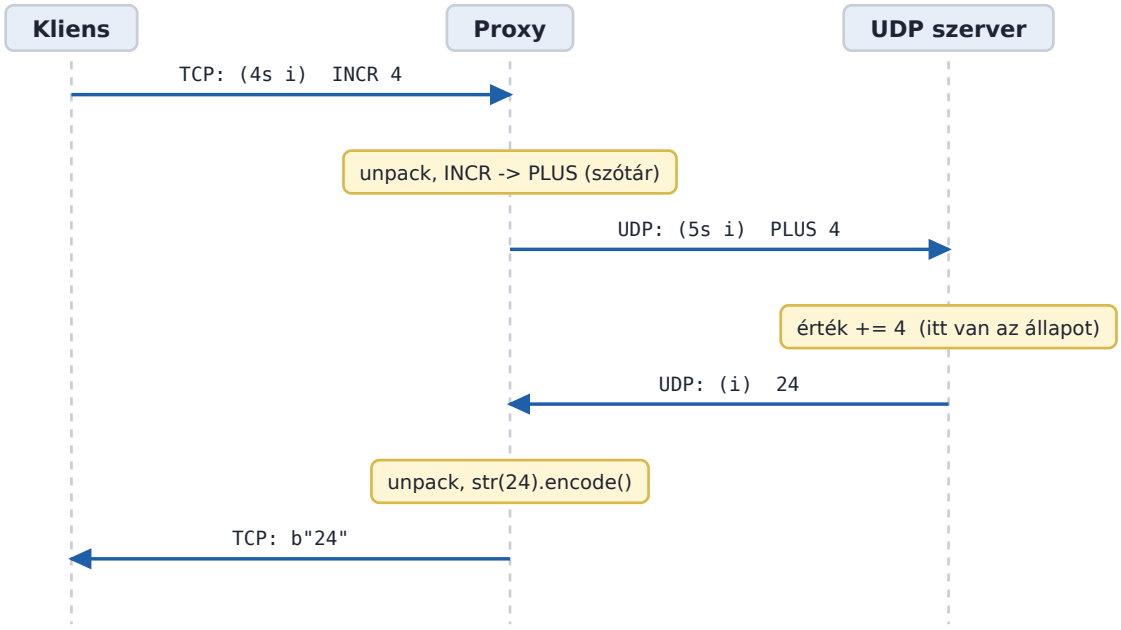
```
def tcp_ertek():
    with socket.create_connection(("localhost", TCP_PORT)) as sock:
        sock.sendall(TCP_CMD.pack(b"INCR", 0))
        return int(sock.recv(1024))                # b"19" -> 19 (az int() bájtot is elfogad)

def udp_ertek():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.settimeout(2)
        sock.sendto(UDP_CMD.pack(b"PLUS", 0), ("localhost", UDP_PORT))
        data, _ = sock.recvfrom(1024)
    return REPLY.unpack(data)[0]                   # tuple! kell a [0]

hibazott = tcp_ertek() != udp_ertek()
```

6/c Proxy (`p5_tcp_proxy.py`)

A TCP szerver most **nem tárol értéket**: a kliensek parancsait átfordítja, továbbítja az UDP szervernek, és az UDP szerver válaszát küldi vissza a kliensnek.




```
FORDITAS = {"IN": "PUSH", "INCR": "PLUS", "DECR": "MINUS"}       # a feladat magja

udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)           # EGY UDP socket, ciklus előtt
udp.settimeout(2)
# ... select-es TCP szerver, mint a P3-ban; a kliens üzenetének feldolgozása:
nev, x = TCP_CMD.unpack(data)                                    # 1) kliens formátuma (4s i)
udp_nev = FORDITAS.get(nev.rstrip(b"\x00").decode())             # 2) átfordítás
udp.sendto(UDP_CMD.pack(udp_nev.encode(), x), UDP_ADDR)          # 3) UDP formátum (5s i)
valasz, _ = udp.recvfrom(1024)                                   # 4) válasz megvárása
(ertek,) = REPLY.unpack(valasz)                                  # 5) kicsomagolás (i)
s.sendall(str(ertek).encode())                                   # 6) a kliensnek bytesztring
```

Tipikus hibák

- **három különböző formátum**: `4s i` (kliens felől), `5s i` (UDP felé), `i` (UDP válasz). A bytesztring csak a kliens felé megy.
- az UDP socketet **nem kell** a select listába tenni: a választ rögtön a kérés után megvárjuk.
- az UDP socket a ciklus előtt jön létre, nem minden üzenetnél újra.
- az átfordítás szótárában hiányzik egy parancs, vagy az ismeretlen parancsnál `KeyError` dobódik (használj `.get`).
- `settimeout` nélkül, ha az UDP szerver nem válaszol, a proxy örökre lefagy.

7. Hibaüzenet-kereső

Ha valami nem megy, innen indulj: üzenet → valószínű ok → teendő.

Mit látsz	Valószínű ok	Teendő
`struct.error: unpack requires a buffer of 12 bytes`	rossz méretet olvastál vagy a méretet kézzel írtad be	mindig `Struct.size`; TCP-n `recv_exact`; a kitöltés miatt `5s i` = 12
`struct.error: argument for 's' must be a bytes object`	str-t adtál a pack-nek	`.encode()`
`TypeError: a bytes-like object is required, not 'str'`	str-t küldtél a sendall-nak	`.encode()`, pl. `str(ertek).encode()`
`ConnectionRefusedError` / WinError 10061	nem fut a szerver, rossz a port, vagy még nem indult el	indítsd előbb a szervert; ellenőrizd a portot
`OSError: Address already in use` (Linux 98, Mac 48, Windows 10048)	a port még foglalt (régi futás)	`SO_REUSEADDR`; zárd be a régi folyamatot
a program **megáll**, nem történik semmi	a `recv` választ vár, ami nem jön	rossz méret? rossz formátum? UDP-nél `settimeout`
`socket.timeout`	nem jött válasz időben	futott a szerver? jó porton? megkapta a kérést?
`ConnectionResetError` / WinError 10054	a másik oldal hirtelen bontott	`try/except ConnectionResetError`, kezeld bontásként
a válasz tele van `\x00` bájtokkal	kipótolt `s` mező	`.rstrip(b"\x00")`
`UnicodeDecodeError`	olyan bájtokat dekódolsz, amik nem szöveg (pl. egy szám bájtjai)	csak az `s` mezőt `.decode()`-old
a szerver 100%-on pörög, folyamatosan „bontott kliens”-t ír	a bontott socket benne maradt az `inputs`-ban	`inputs.remove(s)` + `s.close()`
`KeyError: 'XYZ'`	ismeretlen parancs a szótárban	`FORDITAS.get(nev)` + `ERR` válasz
az érték minden kérésnél újra 0	az állapotváltozót a ciklusban hoztad létre	a ciklus **elé** tedd

8. Macen gyakorolsz, Windowson írod a ZH-t

Téma	Mac	Windows (ZH)
Python indítása	`python3 fajl.py`	`python fajl.py` vagy `py fajl.py`
Ctrl+C	megszakítja a blokkoló hívást	az `accept`-et/`recv`-et nem mindig szakítja meg: ezért van az órai `square_server.py`-ban `settimeout(1.0)` (megjegyzés: „Windowson a Ctrl+C csak így szakítja meg"). A select-es ciklusban az 1.0 timeout ugyanezt oldja meg
Bontó kliens	üres adat (`b""`)	néha `ConnectionResetError`: fogd el, kezeld bontásként
`SO_REUSEADDR`	újraindításkor feloldja a foglalt portot	másképp működik: két szerver is bindolhat ugyanarra a portra. Ha furcsa válaszokat kapsz, nézd meg a Feladatkezelőben, nem fut-e még egy régi `python.exe`
Sorvég a fájlokban	`\n`	`\r\n`: használj `.splitlines()` vagy `.strip()`-et, ne `.split("\n")`-t
Fájl megnyitása	`open(fajl, encoding="utf-8")`	ugyanez; az útvonal a **futtatás mappájához** képest értendő (cwd)
Tűzfal	-	első indításkor a Windows rákérdezhet a hálózati engedélyre: Engedélyezés. Localhoston általában nem kell
Ékezetes `print`	rendben	a konzol néha gondot okoz: a ZH-n a kiírásokat egyszerűen tartsd ékezetmentesen

TIPP A kódod két ponton ugyanúgy működik mindkét rendszeren, ha betartod: **nem** írsz be kézzel méreteket (`Struct.size`), a fájlokat `encoding="utf-8"`-tel nyitod meg, és a szerverciklusban mindig van timeout.

9. A saját segédleted: mi kerüljön bele?

A ZH nyílt könyves, de a lényeg: **a már kipróbált, működő saját kódod** legyen kéznél, ne netes példák. Ajánlott felépítés (1-2 oldal):

- **struct kisokos**: formátumjelek, méretek táblázata, a kitöltés-csapda (`"5s i"` = 12), `encode / rstrip / decode`.
- **recv_exact**: a 8 soros függvény készen.
- **5 sablon**, mindegyik a saját megoldásodból: TCP kliens, UDP kliens, select-es TCP szerver, UDP szerver, proxy.
- **A ZH-s ellenőrzőlista** (0. fejezet, első 3 perc): ki a szerver, TCP/UDP, formátumok, válasz típusa.
- **Hibaüzenet-kereső**: a 7. fejezet leglényegesebb 5 sora.

FONTOS Az AI-val készült segédlet engedélyezése **nincs az anyagban**: kérdezd meg a gyakvezetőt. A biztonságos megoldás, hogy az F1-F5 feladatokat te magad írod meg, és ezekből a saját kódjaidból állítod össze a segédletet: ez a legjobb tanulás is.

10. Önellenőrző kérdések

Először válaszolj fejből, aztán nézd meg a megoldást.

1. Mekkora a `struct.Struct("5s i").size`?

Válasz: 12 (5 + 3 kitöltő bájt + 4).

2. Mi a baj ezzel: `sock.sendall("IN")`?

Válasz: str-t küldesz, bájt kell. A ZH-n struct.pack-kel, `.encode()`-dal vagy `b"IN"`-nel.

3. Miért kell TCP-nél recv_exact?

Válasz: A TCP bájtfolyam, nincs üzenethatár: a `recv(n)` kevesebbet is adhat n-nél.

4. Mely hívásokat NEM használod UDP-nél a TCP-hez képest?

Válasz: `listen`, `accept`, `connect` (és a recv_exact-et).

5. A select visszaadja a figyelő (server) socketet. Mit jelent ez?

Válasz: Új kliens akar csatlakozni: `accept` jön, és a kliens bekerül az `inputs` listába.

6. Mit jelent, ha `recv_exact` üres bájtsort ad vissza?

Válasz: A kliens bontott: `inputs.remove(s)` és `s.close()`.

7. Mit ad a `struct.unpack("4s", b"amla")`?

Válasz: `(b'amla',)`, egy tuple. Kell a `[0]` vagy `(x,) = ...`.

8. A proxynál miért nem kell az UDP socketet a select listába tenni?

Válasz: A választ rögtön a kérés után megvárjuk (`recvfrom`), nem külön eseményként kezeljük.

9. Mi megy a TCP kliensnek a select-es szerverből (`IN 20` után), és mi az UDP kliensnek (`PUSH 20` után)?

Válasz: TCP: bytesztring `b"20"`. UDP: struct `i` (4 bájt).

Ha mind a kilenc megvan, és az F1-F5 teszt zöld, a ZH-sablonok a kezedben vannak.